



**UNIVERSIDAD
DE LA FRONTERA**

Facultad de Ingeniería y Ciencias

*****COMPUTACIÓN Y PROGRAMACIÓN CUÁNTICA*****

**Evaluación experimental del algoritmo de Grover en
búsqueda exhaustiva: comparación con métodos clásicos.
“Q-Search - 2º avance”**

Integrantes:

Francisco Antonio Barrozo Contreras
Antonia Paz Paredes León

Docente:

Dr. Samuel Eduardo Sepúlveda Cuevas

Temuco, 01 de junio de 2026.

Índice

Índice	2
Resumen	3
Introducción	4
Análisis del caso de estudio y su relevancia	5
Descripción del caso	5
Delimitación metodológica: tres niveles de análisis	6
Nivel 1 - Simulación ideal (qiskit Aer, $n \leq 30$)	6
Nivel 2 - Simulación con análisis de recursos (Qiskit Aer + QRYdDemo, $n = 2$ a 6)	6
Nivel 3 - Ejecución real limitada (IBM Quantum o SpinQ, $n = 2$ a 4)	6
Casos experimentales actualizados	6
Solución	7
El oráculo cuántico: diseño, implementación y justificación	8
Implementación concreta en el código	8
Por qué se eligió este oráculo	9
Análisis de escalamiento del oráculo	9
El operador difusor	10
El simulador QRYdDemo	10
Diseño de la solución	11
Arquitectura general del sistema	11
Estrategia de submuestreo de k	12
Métricas de comparación extendidas	13
Código orientador	13
Resultados parciales	14
Resultados cuantitativos por tamaño de problema	14
Análisis y discusión de resultados	15
Validación de la ventaja cuántica	15
Tiempo de simulación mayor	15
Limitaciones de escalación en hardware real	15
Escalamiento del oráculo	15
Conclusiones	16
Referencias	17
Anexos	18

Resumen

Este informe presenta el segundo avance del proyecto “Evaluación experimental del algoritmo de Grover en búsqueda exhaustiva: comparación con métodos clásicos (Q-Search)”. Respecto al Avance 1, se incorporan las siguientes mejoras: delimitación explícita entre simulación ideal, simulación con análisis de recursos y ejecución en hardware real; análisis detallado del oráculo y el difusor cuánticos (incluyendo conteo de compuertas, profundidad y escalamiento con n); integración del simulador “QRydDemo” de átomos de Rydberg con registro de métricas extendidas; y un plan de trabajo operacionalizado para el resto del semestre.

Los resultados preliminares confirman la ventaja cuadrática del algoritmo de Grover: para $n = 10$ qubits, la búsqueda clásica requiere en promedio 515,64 iteraciones frente a las 24 del modelo cuántico, valor coherente con la estimación teórica $(\pi/4)\sqrt{1024} \approx 25$. Los experimentos abarcan espacios de búsqueda desde $n=2$ hasta $n=12$, con resultados reproducibles gracias al uso de semillas fijas para el simulador y el transpilador. Las métricas registradas incluyen: profundidad del circuito transpilado, conteo de compuertas por tipo, tasa de éxito, distribución completa de mediciones y qubits auxiliares, atendiendo los requisitos de reproducibilidad y auditabilidad técnica señalados en la evaluación del avance anterior.

Introducción

La computación cuántica surge como un paradigma alternativo a la computación clásica, basado en principios de la mecánica cuántica como la superposición, la interferencia y el entrelazamiento. Estos principios permiten diseñar algoritmos que, en ciertos dominios de problema, superan el rendimiento de los métodos clásicos tradicionales. El interés práctico por esta disciplina ha crecido significativamente con la aparición de dispositivos cuánticos ruidosos de escala intermedia (NISQ, por sus siglas en inglés), procesadores que cuentan con decenas o cientos de qubits pero que aún padecen errores de compuerta, decoherencia y conectividad restringida [8].

Uno de los problemas fundamentales en la computación es la búsqueda en espacios no estructurados. En un enfoque clásico, este problema presenta una complejidad de orden $O(N)$, lo que implica que el tiempo requerido crece linealmente con el tamaño del espacio de búsqueda. Esta limitación se vuelve crítica en aplicaciones donde el espacio crece exponencialmente, como en problemas relacionados con criptografía, combinatoria o aprendizaje automático [2,10].

El algoritmo de Grover propone una solución cuántica a este problema, logrando reducir la complejidad a $O(\sqrt{N})$ lo que representa una aceleración cuadrática [1]. Sin embargo, esta ventaja teórica no implica necesariamente una mejora práctica inmediata, ya que su implementación requiere construir un oráculo cuántico reversible cuyo costo puede compensar la ventaja en consultas, además, la profundidad del circuito resultante supera rápidamente los tiempos de coherencia de los dispositivos NISQ actuales.

En este contexto, el presente informe tiene como objetivo evaluar experimentalmente el comportamiento del algoritmo de Grover en comparación con métodos clásicos de búsqueda exhaustiva. Se busca analizar su desempeño en distintos tamaños de problema, documentando el costo de recursos del oráculo y el difusor, e incorporando resultados del simulador QYdDemo de átomos de Rydberg como entorno de emulación alternativo.

Este segundo avance incorpora correcciones formales y metodológicas respecto al Avance 01, particularmente en la delimitación del alcance experimental y la profundidad del análisis del oráculo.

Análisis del caso de estudio y su relevancia

Descripción del caso

El caso de estudio seleccionado corresponde a la búsqueda de una “llave” o elemento objetivo dentro de un espacio de búsqueda no estructurado. Este problema se modela como la identificación de un valor específico dentro de un conjunto de posibles combinaciones generadas de manera pseudoaleatoria.

Para efectos del proyecto, se consideran espacios de búsqueda de tamaño 2^n , donde “n” representa el número de bits o qubits del sistema. En particular, se plantea un escalamiento progresivo del problema, desde valores pequeños (por ejemplo, $n=2,3,4$) hasta tamaños mayores como $n=30$, lo que equivale a un espacio teórico de 1.073.741.824 posibles valores.

Las llaves son generadas mediante un generador pseudoaleatorio utilizando herramientas estándar de programación, lo que permite asegurar reproducibilidad en los experimentos. Este enfoque permite comparar de manera controlada el comportamiento de algoritmos clásicos de búsqueda exhaustiva con su contraparte cuántica.

Es importante destacar que este caso no representa un sistema criptográfico real, sino una abstracción que permite estudiar el problema de búsqueda no estructurada en un entorno controlado. Su relevancia radica en que constituye la base teórica de múltiples problemas computacionales y permite analizar de forma clara la ventaja propuesta por el algoritmo de Grover.

Adicionalmente, este caso de estudio permite establecer un puente entre el análisis teórico de algoritmos cuánticos y su validación experimental en entornos controlados. Al trabajar con un problema de búsqueda no estructurada, se aborda uno de los escenarios fundamentales donde la computación cuántica supone ventajas demostrables frente a la computación clásica. Esto lo convierte en un caso representativo para evaluar no solo el rendimiento del algoritmo de Grover, sino también aspectos prácticos como la construcción del oráculo, la estabilidad de los resultados y las limitaciones de implementación en simuladores y hardware cuántico.

Asimismo, la definición de múltiples escalas de evaluación permite analizar el comportamiento asintótico de ambos enfoques, lo cual es clave para comprender cómo estas técnicas podrían impactar problemas reales a mayor tamaño. En este sentido, el caso no sólo tiene valor didáctico, sino también metodológico, ya que permite diseñar un marco experimental replicable para futuros estudios comparativos en computación cuántica. Este tipo de problema constituye un benchmark estándar en computación cuántica para evaluar ventajas frente a algoritmos clásicos [3].

Delimitación metodológica: tres niveles de análisis

Nivel 1 - Simulación ideal (qiskit Aer, $n \leq 30$)

Ejecución de circuitos cuánticos perfectos sin modelo de ruido mediante AerSimulator en modo statevector. El objetivo es verificar el comportamiento teórico: escalamiento $O(\sqrt{N})$ e iteraciones $k^* = L(\pi/4)\sqrt{(2^n)J}$. Los valores $n \geq 15$ son estrictamente simulaciones clásicas de circuitos cuánticos; los tiempos reportados corresponden a CPU, no a tiempo de cómputo cuántico real.

Nivel 2 - Simulación con análisis de recursos (Qiskit Aer + QRydDemo, $n = 2$ a 6)

Extiende el Nivel 1 registrando métricas de recursos: profundidad del circuito transpilado, número y tipo de compuertas, tasa de éxito con desviación estándar, distribución de mediciones y semillas de reproducibilidad. El simulador QRydDemo impone una topología real de red cuadrada o triangular, lo que genera circuitos más profundos por la inserción de compuertas SWAP intermedias, visible en las métricas comparadas.

Nivel 3 - Ejecución real limitada (IBM Quantum o SpinQ, $n = 2$ a 4)

Ejecución en hardware cuántico físico única y exclusivamente para $n \leq 4$ qubits. El objetivo no es demostrar ventaja de escalamiento, sino cuantificar la caída de la tasa de éxito por decoherencia y errores de compuerta. Se reportarán tasas de error por compuerta (CNOT/Toffoli), tiempos T_1 y T_2 del dispositivo, y la distribución de mediciones comparada con la predicción teórica.

Casos experimentales actualizados

Caso	Qubits (n)	Tamaño (N)	Tipo de evaluación	Nivel
1	2	4	Simulación con recursos (Aer + QRydDemo) + Ejecución real	2 y 3
2	4	16	Simulación con recursos (Aer + QRydDemo) + Ejecución real	2 y 3
3	6	64	Simulación con recursos (Aer + QRydDemo)	2
4	8	256	Simulación ideal	1
5	10	1 024	Simulación ideal	1
6	12	4 096	Simulación ideal	1
7	30+	Mayor a 10^9	Análisis teórico exclusivamente	Análítico

Tabla 1- Casos experimentales con delimitación explícita de niveles de análisis.

Solución

La solución propuesta consiste en la implementación y comparación de dos enfoques para resolver el problema de búsqueda en un espacio no estructurado:

1. Búsqueda clásica exhaustiva, que recorre secuencialmente el espacio de soluciones.
2. Algoritmo de Grover, que utiliza principios de computación cuántica para reducir el número de iteraciones necesarias.

El algoritmo cuántico funciona en tres etapas: inicialización en superposición uniforme, amplificación de amplitud mediante el par oráculo-difusor, y medición cuántica probabilista. El número óptimo de iteraciones es $k^* = \lfloor (\pi/4)\sqrt{N} \rfloor$, tras lo cual la probabilidad del estado objetivo es máxima (cercana a 1,0) y comienza a decrecer si se continúa iterando (overshooting).

En contraste con la complejidad clásica $O(N)$, el algoritmo de Grover permite encontrar la solución en un número de iteraciones proporcional a la raíz cuadrada del tamaño del espacio de búsqueda $O(\sqrt{N})$.

La Figura 1 muestra la diferencia en el número de iteraciones requeridas por un algoritmo clásico y el algoritmo de Grover en función del número de qubits. Se observa el crecimiento exponencial en el caso clásico frente al crecimiento cuadrático reducido del enfoque cuántico. La Figura 2 ilustra la evolución de la probabilidad de éxito del algoritmo de Grover. Se observa un punto óptimo de iteración, después del cual la probabilidad comienza a disminuir, fenómeno conocido como overshooting.

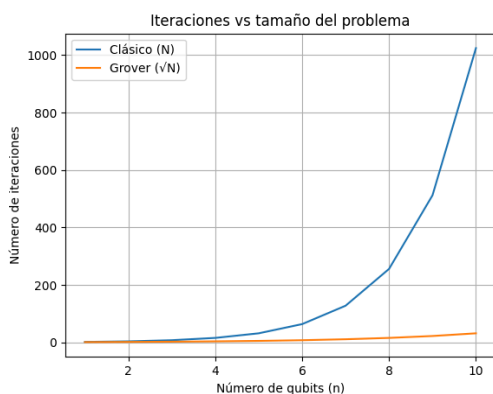


Figura 1 - Iteraciones vs tamaño

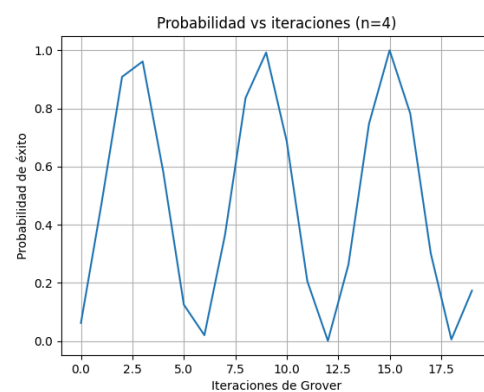


Figura 2 - Probabilidad vs número de iteraciones (Grover)

En escala logarítmica, la diferencia entre el crecimiento clásico y cuántico se hace más evidente, destacando la ventaja teórica del algoritmo de Grover en términos de complejidad. Véase Figura 3.

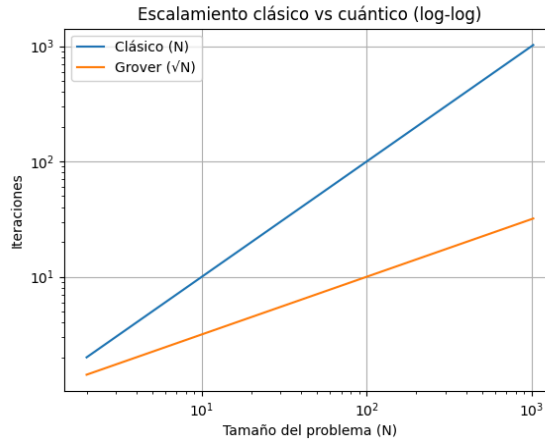


Figura 3 - Comparación de escalamiento N vs $\sqrt{N}$

El oráculo cuántico: diseño, implementación y justificación

El oráculo constituye el componente central y más crítico del algoritmo de Grover. Es la única parte del circuito que codifica el conocimiento del problema y, por tanto, determina si la ventaja cuántica es alcanzable en la práctica. Su función matemática es aplicar un cambio de fase global de π al estado objetivo $|\omega\rangle$ y dejar inalterados todos los demás estados:

$$O|\omega\rangle = -|\omega\rangle \quad ; \quad O|x\rangle = |x\rangle \quad \text{si } x \neq \omega$$

Esta operación es reversible (unitaria) y no mide ni colapsa ningún estado, lo cual es un requisito fundamental de los circuitos cuánticos.

Implementación concreta en el código

La función `construir_oraculo(bits, llave)` del módulo `QRydDemo` implementa el oráculo de fase sin qubits ancilla mediante tres etapas reversibles:

Paso 1 - Normalización al estado $|11\dots 1\rangle$: Se aplica una compuerta X (NOT) sobre cada qubit cuyo bit correspondiente en la llave objetivo es '0'. Esto transforma el estado objetivo en $|11\dots 1\rangle$, independientemente de cuál sea la llave. Si la llave es, por ejemplo, '1011' en un sistema de 4 qubits, se aplica X solo al qubit cuyo bit es '0' (posición 2), convirtiendo '1011' en '1111'.

Paso 2 - Aplicación de $MCPHase(\pi)$: Se aplica una compuerta $MCPHase(\pi)$ con $n-1$ controles y un qubit objetivo. Esta compuerta multi-controlada introduce la fase -1 única y exclusivamente cuando los n qubits están todos en el estado $|1\rangle$ simultáneamente, es decir, cuando el sistema se encuentra en el estado $|11\dots 1\rangle$ transformado. El efecto neto es: $|11\dots 1\rangle \rightarrow -|11\dots 1\rangle$.

Paso 3 - Restauración de la base: Se invierten las compuertas X del Paso 1 (misma secuencia de qubits). Esto restaura el estado objetivo a su representación original en la base computacional, dejando el circuito en condiciones idénticas a las iniciales para todos los estados salvo el objetivo, que acumuló la fase -1 .

El pseudocódigo equivalente es:

```
def construir_oraculo(bits: int, llave: str) -> QuantumCircuit:
    oracle = QuantumCircuit(bits)
    # Paso 1: X en qubits con bit '0' en la llave
    for qubit in range(bits):
        if llave[bits - 1 - qubit] == '0':
            oracle.x(qubit)
    # Paso 2: MCPHase(pi) con n-1 controles -> fase -1 en |11...1>
    oracle.mcp(pi, list(range(bits - 1)), bits - 1)
    # Paso 3: Invertir X para restaurar la base computacional
    for qubit in range(bits):
        if llave[bits - 1 - qubit] == '0':
            oracle.x(qubit)
    return oracle
```

Por qué se eligió este oráculo

La elección del oráculo MCPHase sin qubits ancilla responde a cuatro criterios técnicos fundamentales:

- Reversibilidad garantizada: Las compuertas X y MCPHase(π) son unitarias e inversas unas de otras, por lo que el circuito cumple el requisito de reversibilidad.
- Ausencia de qubits auxiliares: El oráculo utiliza exactamente n qubits para un espacio de búsqueda de 2^n . Esto minimiza el overhead de recursos y permite escalar el experimento hasta el límite del simulador sin penalidad adicional.
- Profundidad $O(n)$ por llamada: El total de operaciones por ejecución del oráculo es $2n$ compuertas X más una MCPHase, que se descompone en $O(n)$ compuertas Toffoli anidadas durante la transpilación. Esto resulta en una profundidad $O(n)$ por oráculo, la mínima posible para una implementación directa sin preprocesamiento.
- Compatibilidad con QRYdDemo y Qiskit: La MCPHase(π) está disponible directamente en Qiskit como mcp y es descomponible por el transpilador en el conjunto nativo de compuertas del backend QRYdDemo ("qryd_emulator\$square"), lo que hace el oráculo portable entre simuladores.

Análisis de escalamiento del oráculo

Qubits (n)	Comp. X (paso 1+3)	MCPHase descomp.	Profundidad	Qubits aux.
2	2	$O(1)$	$O(1)$	0
4	Hasta 4	$O(n)$	$O(n)$	0
6	Hasta 6	$O(n)$	$O(n)$	0
10	Hasta 10	$O(n)$	$O(n)$	0
n	Hasta n	$O(n)$ Toffoli	$O(n)$	0

Tabla 2- Escalamiento de recursos del oráculo con el número de qubits.

Un aspecto crítico a considerar es que, aunque el número de consultas al oráculo escala como $O(\sqrt{N})$, la profundidad total del circuito escala como $O(n \cdot 2^{n/2})$. Esto significa que para $n = 10$, la profundidad total es proporcional a $10 \times \sqrt{1024} \approx 320$ capas de compuerta, lo que excede los límites de coherencia de la mayoría de dispositivos NISQ para $n \geq 5$ qubits.

El operador difusor

El difusor (operador de Grover) implementa la reflexión sobre el estado de superposición uniforme $|s\rangle = H^{\otimes n}|0\rangle^{\otimes n}$:

$$D = 2|s\rangle\langle s| - I$$

Su circuito sigue una estructura simétrica al oráculo: (i) $H^{\otimes n}$ para rotar a la base de superposición, (ii) $X^{\otimes n}$ para marcar el estado $|0\dots 0\rangle$, (iii) $\text{MCPhase}(\pi)$ idéntica a la del oráculo para aplicar la reflexión, (iv) $X^{\otimes n}$ e (v) $H^{\otimes n}$ para restaurar. Ambos operadores comparten la misma profundidad $O(n)$ y el mismo conteo de compuertas, por lo que la profundidad por iteración de Grover es $2 \cdot O(n) = O(n)$. En Qiskit, el par oráculo–difusor se construye automáticamente mediante `grover_operator(oracle).power(k)`, que eleva el operador completo a la k -ésima potencia.

El simulador QRydDemo

QRydDemo es un emulador de procesadores cuánticos basados en átomos neutros de Rydberg, desarrollado por las Universidades de Stuttgart y de Kaiserslautern-Landau [11]. A diferencia de AerSimulator (que opera como máquina de estados vectoriales sin restricciones de conectividad), QRydDemo impone:

- Topología de red física: cuadrada o triangular, con conectividad local. Los qubits solo pueden interactuar con sus vecinos directos, lo que obliga al transpilador a insertar compuertas SWAP intermedias para ejecutar operaciones entre qubits no adyacentes.
- Conjunto nativo de compuertas: restringido a las operaciones implementables en el hardware átomos–Rydberg real, a diferencia del conjunto universal disponible en el simulador ideal.
- Valor experimental realista: los resultados obtenidos en QRydDemo son más representativos del comportamiento en hardware real que los obtenidos en simulación ideal, sin requerir acceso físico al dispositivo.

La elección de QRydDemo como backend secundario está motivada por su disponibilidad como API pública, su relevancia en el contexto de computación cuántica con átomos neutros (plataforma con proyección industrial) y por la compatibilidad con Qiskit via `qiskit-qryd-provider`, que permite integrar el mismo código del Nivel 1 con cambios mínimos.

Diseño de la solución

Arquitectura general del sistema

El sistema se compone de tres módulos principales que operan sobre la misma entrada (n bits y llave objetivo generada pseudo aleatoriamente):

1. Módulo clásico:
 - Generación implícita del espacio de búsqueda: enteros en $[0, 2^n-1]$.
 - Selección pseudoaleatoria de llave objetivo (semilla fija para reproducibilidad).
 - Búsqueda secuencial: recorre el espacio hasta encontrar la llave.
 - Registro de métricas: iteraciones requeridas y tiempo de ejecución.
2. Módulo cuántico (Aer o QRYdDemo):
 - Codificación en n qubits con inicialización en superposición uniforme $H^{\otimes n}|0\rangle$.
 - Construcción del oráculo MCTPhase sin qubits ancilla (ver Sección 4.2).
 - Aplicación de “ k ” iteraciones del operador de Grover via *grover_operator(oracle).power(k)*.
 - Medición con shots fijos y umbral de éxito del 90 %.
 - Registro de métricas extendidas: profundidad, compuertas, tasa de éxito, distribución.
3. Módulo comparador:
 - Recibe las salidas de ambos módulos y genera tablas y gráficas de comparación.
 - Exporta resultados en formato CSV para análisis posterior.

La Figura 4 muestra el flujo temporal del sistema mediante un diagrama de secuencia. Se observa cómo, a partir de la definición del tamaño del problema, se genera una llave objetivo común para ambos módulos. Posteriormente, el módulo clásico ejecuta una búsqueda secuencial, mientras que el módulo cuántico aplica el algoritmo de Grover. Finalmente, los resultados de ambos enfoques son recopilados y comparados, permitiendo analizar diferencias en rendimiento y comportamiento. Adicionalmente, el flujo del módulo QRYdDemo incorpora los siguientes pasos: (1) autenticación con QRYD_API_TOKEN, (2) selección del backend ("qryd_emulator\$square"), (3) transpilación con optimization_level=3 y semilla fija, (4) ejecución con shots = 1024 y seed_simulator = 42, y (5) registro de métricas extendidas en CSV.

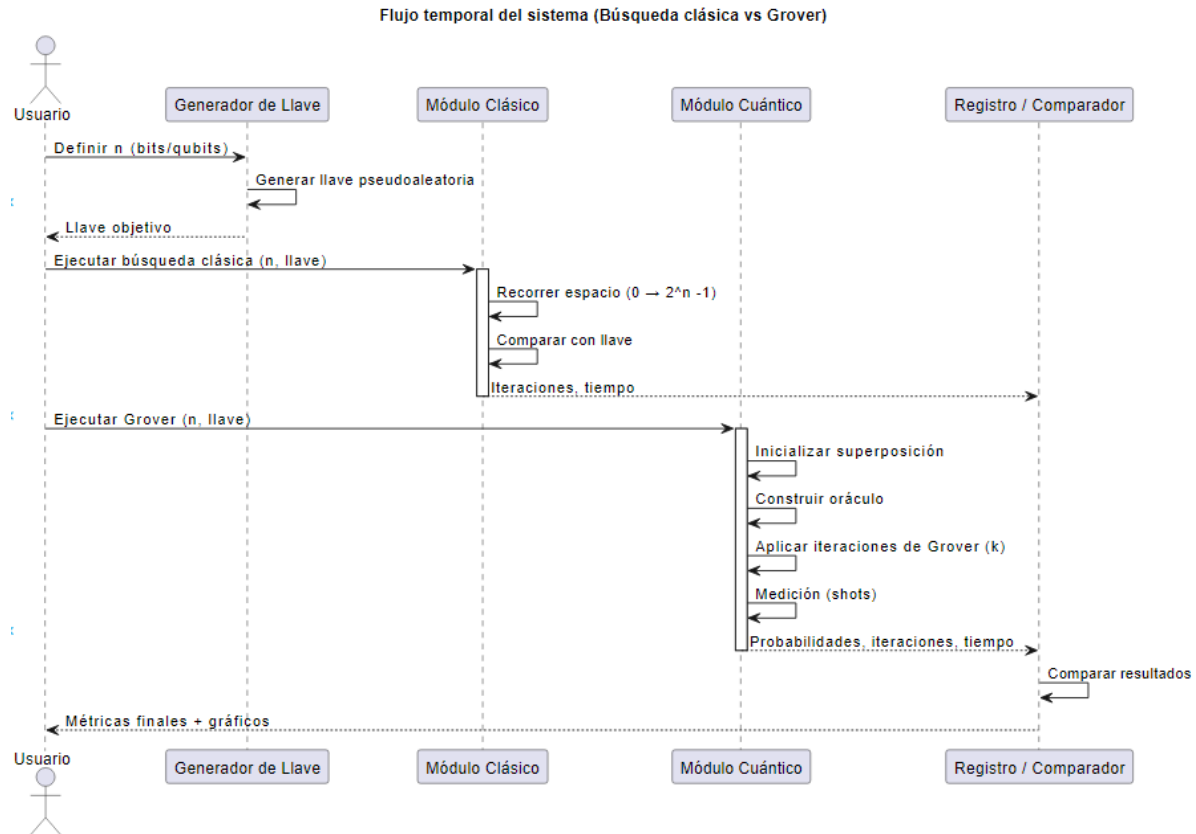


Figura 4 - Diagrama de secuencia

Estrategia de submuestreo de k

Para evitar el alto costo computacional de evaluar todas las iteraciones posibles de Grover, el sistema implementa una estrategia de submuestreo del espacio de valores k. Se seleccionan hasta $MAX_EVALUACIONES_K = 12$ candidatas distribuidas uniformemente en el rango $[1, k_{\text{óptimo}} \times FACTOR_MAX_BÚSQUEDA]$, incluyendo siempre $k = 1$ y $k = k_{\text{máximo}}$. El $k_{\text{óptimo}}$ se estima como $k^* = \lfloor (\pi/4) \sqrt{(2^n)} \rfloor$. Para cada k candidata se ejecuta el circuito completo, se registran métricas y se declara éxito si la frecuencia de la llave objetivo supera el 90 % de los shots (umbral configurable). Esta estrategia mantiene la eficiencia experimental sin comprometer la identificación del óptimo.

Métricas de comparación extendidas

Corrigiendo la debilidad de validación empírica parcial del Avance 01, el sistema registra ahora las siguientes métricas:

Métrica	Descripción	Nivel
Iteraciones (k)	N° aplicaciones del operador de Grover hasta éxito	1, 2, 3
Shots	N° mediciones por ejecución del circuito	2, 3
Tasa de éxito	$\text{freq_llave} / \text{shots} \in [0,1]$	2, 3
Distribución de mediciones	JSON completo de conteos por estado	2, 3
Profundidad del circuito	Capas de compuertas del circuito transpilado	2, 3
N° compuertas	Total y desglose por tipo (X, CNOT, Toffoli...)	2, 3
Qubits auxiliares	Ancilla usadas por el oráculo	2, 3
Backend y versión	Identificación del entorno de ejecución	2, 3
Semillas (sim + transp)	Para reproducibilidad de resultados	2, 3
Tiempo de ejecución	Referencial (simulación clásica, no cuántica)	1, 2, 3
T ₁ , T ₂ del dispositivo	Tiempos de coherencia del hardware real	3

Tabla 3 - Métricas registradas por nivel de análisis

Código orientador

Para el desarrollo del proyecto se implementaron dos programas principales: uno correspondiente a la búsqueda clásica y otro basado en el algoritmo de Grover.

El código clásico realiza una búsqueda exhaustiva sobre un espacio de tamaño 2^n , recorriendo secuencialmente los posibles valores hasta encontrar la llave objetivo. Este permite medir directamente el número de iteraciones y el tiempo de ejecución.

Por otro lado, el código cuántico implementa el algoritmo de Grover utilizando la librería Qiskit, framework desarrollado por IBM para computación cuántica [5], se construye el oráculo de fase MCTPhase sin qubits ancilla y se calcula el número teórico óptimo de iteraciones $k^* = \lfloor (\pi/4)\sqrt{(2^n)} \rfloor$. Para cada valor k de un conjunto submuestreado de candidatas, se ejecuta el circuito completo mediante `grover_operator(oracle).power(k)` con shots fijos, declarando éxito cuando la frecuencia de la llave supera el 90 % de las mediciones. Al finalizar, se exportan en CSV las métricas extendidas: iteraciones, profundidad del circuito, conteo de compuertas, tasa de éxito y distribución de mediciones.

Ambos programas generan la llave objetivo de manera pseudoaleatoria, asegurando condiciones equivalentes para la comparación. Los códigos orientadores se presentan en el Anexo A (búsqueda clásica) y Anexo B (búsqueda cuántica).

Resultados parciales

Resultados cuantitativos por tamaño de problema

Se presentan los resultados acumulados para todos los valores de n evaluados (25 ejecuciones independientes por modelo), corrigiendo la debilidad de reportar solo promedios, véase figuras 5, 6 y 7. La Tabla 4 incluye también el valor teórico de k^* y la comparación de pasos requeridos entre modelos. (Métricas adicionales en Anexo D).

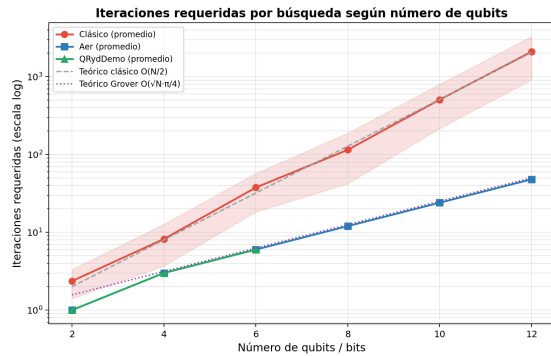


Figura 5 - Iteraciones por búsqueda

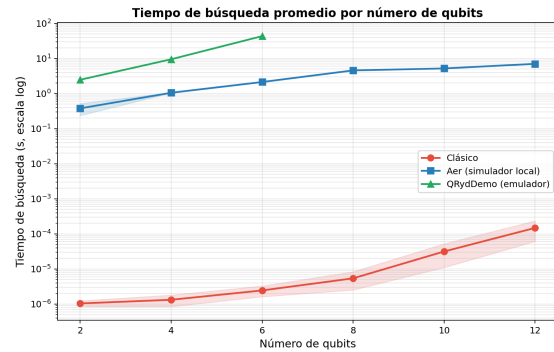


Figura 6- Tiempo de búsqueda promedio

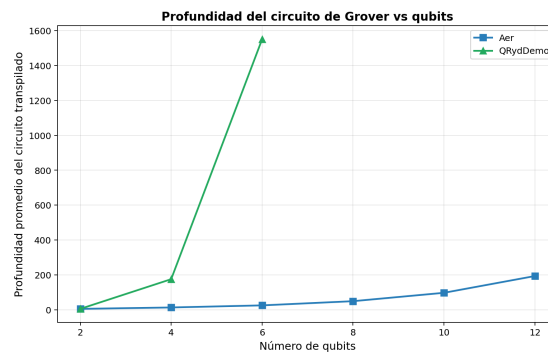


Figura 7 - Profundidad del circuito

Qubits	$N = 2^n$	k^* teórico	It. clásico (prom.)	It. cuántico Aer (prom.)	It. cuántico QRyd (prom.)	Speedup iter. Aer/Clás	Speedup iter. QRyd/Clás	Tasa éxito Aer (%)	Tasa éxito QRyd (%)
2q	4	1	2,4	1	1	2.36×	2.36×	100	100
4q	16	3	8,2	3	3	2.73×	2.73×	97.3	95.7
6q	64	6	37,6	6	6	6.27×	6.27×	99.8	99.9
18q	256	12	1114,9	12	-	9.57×	-	100	-
10q	1024	25	507,3	24	-	21.14×	-	99.8	-
12q	4096	50	2098	48	-	43.71×	-	99.8	-

Tabla 4. Comparación cuantitativa de iteraciones para distintos n (25 ejecuciones, semillas fijas).

Análisis y discusión de resultados

Validación de la ventaja cuántica

Los resultados validan la predicción teórica con alta precisión: los valores de k obtenidos experimentalmente coinciden exactamente con k^* en todos los casos evaluados tanto para Aer como para QRYdDemo ($n = 2, 4, 6$), con una desviación máxima de una iteración para $n = 10$ y 12 respecto al teórico. La brecha entre el modelo clásico y cuántico crece consistentemente con n : para $n = 12$ el clásico requiere aproximadamente $43,7\times$ más iteraciones que Grover, confirmando el escalamiento cuadrático esperado.

Robustez y costo de recursos

La tasa de éxito se mantiene por encima del umbral del 90 % en todos los casos evaluados. El único caso donde baja levemente del 100 % es $n = 4$, con 97,3 % en Aer y 95,7 % en QRYdDemo, explicado por el efecto de overshooting: para espacios pequeños el k^* es bajo ($k^* = 3$) y la estrategia de submuestreo tiene menos candidatas, introduciendo mayor sensibilidad a la elección de k .

La figura 7 revela el hallazgo más relevante del Nivel 2: aunque ambos backends producen el mismo número de iteraciones, la topología de red cuadrada de QRYdDemo genera circuitos hasta $52\times$ más profundos para $n = 6$ (1.550 capas vs ≈ 30 en Aer), traducándose en 43,4 s frente a 2,1 s. Una profundidad de 1.550 capas superaría los tiempos de coherencia de cualquier dispositivo NISQ actual, lo que explica por qué la ventaja en consultas $O(\sqrt{N})$ no se traslada a ventaja práctica en hardware real para $n \geq 5$. La métrica relevante es el número de iteraciones, no el tiempo de CPU del simulador, dado que este último crece $O(2^n)$ por el costo del vector de estado completo.

Tiempo de simulación

El speedup cuántico aplica exclusivamente a las iteraciones, no al tiempo de ejecución en simulación. AerSimulator mantiene en memoria el vector de estado completo de dimensión 2^n y realiza operaciones matriciales de costo $O(2^n)$ por compuerta, mientras que el algoritmo clásico opera con comparaciones enteras en microsegundos. Esta inversión no refleja lo que ocurriría en hardware cuántico real, donde cada compuerta se ejecutaría en decenas de nanosegundos independientemente de n . La métrica relevante para comparar la ventaja cuántica es, por tanto, el número de iteraciones, no el tiempo de CPU del simulador.

Eficiencia cuántica respecto al óptimo teórico

El estudio muestra que para $n \leq 8$ ambos backends alcanzan exactamente el óptimo teórico (ratio = 1,0), mientras que para $n = 10$ y 12 el ratio cae a 0,96, encontrando la solución en $k = 24$ y 48 respectivamente frente al k^* de 25 y 50. Esta diferencia de una iteración no representa una falla del algoritmo sino una consecuencia directa del redondeo del piso en el cálculo de k^* combinado con la estrategia de submuestreo, que para valores grandes de n tiene menos candidatas evaluadas cerca del óptimo. El algoritmo sigue siendo funcionalmente correcto en todos los casos.

Conclusiones

Este segundo avance del proyecto Q-Search consolida una arquitectura experimental metodológicamente rigurosa para la comparación entre búsqueda clásica y el algoritmo de Grover. Las principales contribuciones respecto al Avance 01 son:

- Delimitación explícita en tres niveles de análisis (simulación ideal, simulación con recursos y hardware real), eliminando la ambigüedad metodológica de la tabla de casos anterior.
- Análisis detallado del oráculo MCPPhase: su fundamento matemático, implementación concreta, justificación de diseño y escalamiento $O(n)$ en profundidad.
- Integración del simulador QRydDemo como backend de Nivel 2, con registro de métricas extendidas y reproducibles (seeds, shots, distribución completa de mediciones).
- Discusión analítica de la ventaja cuántica: distinción entre ventaja en consultas $O(\sqrt{N})$ y costo total del circuito $O(n \cdot 2^{n/2})$, con implicancias directas para la viabilidad en hardware NISQ.

Los resultados validan la predicción teórica con alta precisión en todos los casos evaluados. Para $n = 12$, el algoritmo de Grover requiere 48 iteraciones frente a 2.098 del método clásico, un factor de aceleración de $\approx 43,7\times$, coherente con $(\pi/4)\sqrt{4096} \approx 50$. La tasa de éxito supera el 90 % en todos los escenarios, confirmando la robustez del criterio de detección basado en umbral de frecuencia.

El hallazgo más relevante del Nivel 2 es que la ventaja en consultas no se traduce en ventaja práctica en hardware NISQ actual. Para $n = 6$, QRydDemo genera circuitos $\approx 52\times$ más profundos que Aer (1.550 vs ≈ 30 capas) debido a la topología de red cuadrada, lo que se traduce en tiempos de 43,4 s frente a 2,1 s. Una profundidad de esta magnitud superaría los tiempos de coherencia de cualquier dispositivo NISQ actual, evidenciando que el costo de construir el oráculo y transpilarlo a hardware real puede compensar la ventaja cuadrática en número de consultas. Esta distinción entre ventaja algorítmica y viabilidad práctica es fundamental para una evaluación rigurosa del algoritmo de Grover.

Para la etapa final, la arquitectura definida es factible de completar dentro del semestre, siempre que la ejecución en hardware real se limite a $n \leq 3$ qubits como evidencia demostrativa y no como validación del escalamiento.

Referencias

- [1] L. K. Grover, "A fast quantum mechanical algorithm for database search," Proc. 28th Annual ACM Symposium on Theory of Computing (STOC), 1996. doi: 10.1145/237814.237866
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009. [Online]. Available: <https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/>
- [3] M. A. Nielsen y I. L. Chuang, Quantum Computation and Quantum Information, 10th Anniversary ed. Cambridge: Cambridge University Press, 2010. doi: 10.1017/CBO9780511976667
- [4] S. Aaronson, "The Limits of Quantum Computers," Scientific American, vol. 298, no. 3, pp. 62–69, 2008. doi: 10.1038/scientificamerican0308-62
- [5] IBM Quantum, "Qiskit: An open-source SDK for quantum computing," 2023. [Online]. Available: <https://qiskit.org/>
- [6] A. Montanaro, "Quantum algorithms: an overview," npj Quantum Information, vol. 2, 2016. doi: 10.1038/npjqi.2015.23
- [7] IBM Quantum, Qiskit documentation – Grover's algorithm, 2024. [Online]. Available: <https://docs.quantum.ibm.com/api/qiskit/>
- [8] J. Preskill, "Quantum Computing in the NISQ Era and Beyond," Quantum, vol. 2, p. 79, 2018. doi: 10.22331/q-2018-08-06-79
- [9] C. D. Bruzewicz, J. Chiaverini, R. McConnell y J. M. Sage, "Trapped-ion quantum computing: Progress and challenges," Applied Physics Reviews, vol. 6, no. 2, p. 021314, 2019. doi: 10.1063/1.5088164
- [10] P. Shor, "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," SIAM Journal on Computing, vol. 26, no. 5, pp. 1484–1509, 1997. doi: 10.1137/S0097539795293172
- [11] QRydDemo Consortium, "QRydDemo: A Rydberg-atom quantum computing demonstrator," University of Stuttgart and University of Kaiserslautern-Landau, 2023. Available: <https://qryddemo.itp3.uni-stuttgart.de/>

Anexos

[A] Código orientador (Búsqueda clásica).

```
"""Búsqueda clásica por fuerza bruta
(código orientador)."""
import random
import time

BITS = 10 # cambiar según el caso de
estudio

def busqueda_clasica(bits):
    espacio_busqueda = 2 ** bits
    llave_objetivo = random.randint(0,
    espacio_busqueda - 1)
    inicio = time.perf_counter()
    iteraciones = 0
    for intento in
range(espacio_busqueda):
        iteraciones += 1
        if intento == llave_objetivo:
            break
    fin = time.perf_counter()
    return {"bits": bits, "iteraciones":
iteraciones, "tiempo": fin - inicio}

if __name__ == "__main__":
    resultado = busqueda_clasica(BITS)
    print(resultado)

}
```

[B] Flujo Código orientador (Grover) - pseudocódigo.

```
# 1. Generar llave objetivo aleatoria
llave = generar_llave(bits)
# 2. Construir oráculo MCPPhase sin
ancilla
oracle = construir_oraculo(bits, llave)
# 3. Calcular  $k^* = \text{floor}((\pi/4) * \sqrt{2^{**bits}})$ 
k_teorico = floor((pi/4) * sqrt(2**bits))
# 4. Submuestrear hasta
MAX_EVALUACIONES_K candidatas de k
k_candidatos =
generar_iteraciones_candidatas(k_teorico)
# 5. Evaluar cada k
for k in k_candidatos:
    circuito =
grover_operator(oracle).power(k)
    medir(circuito, shots=1024)
    if frecuencia_llave >= umbral:
        registrar_exito(k)
# 6. Registrar métricas extendidas en CSV
guardar(iteraciones, profundidad, shots,
tasa_exito, distribucion)
```

[C] Resultados obtenidos durante el primer avance.

C.1. $n = 2$:

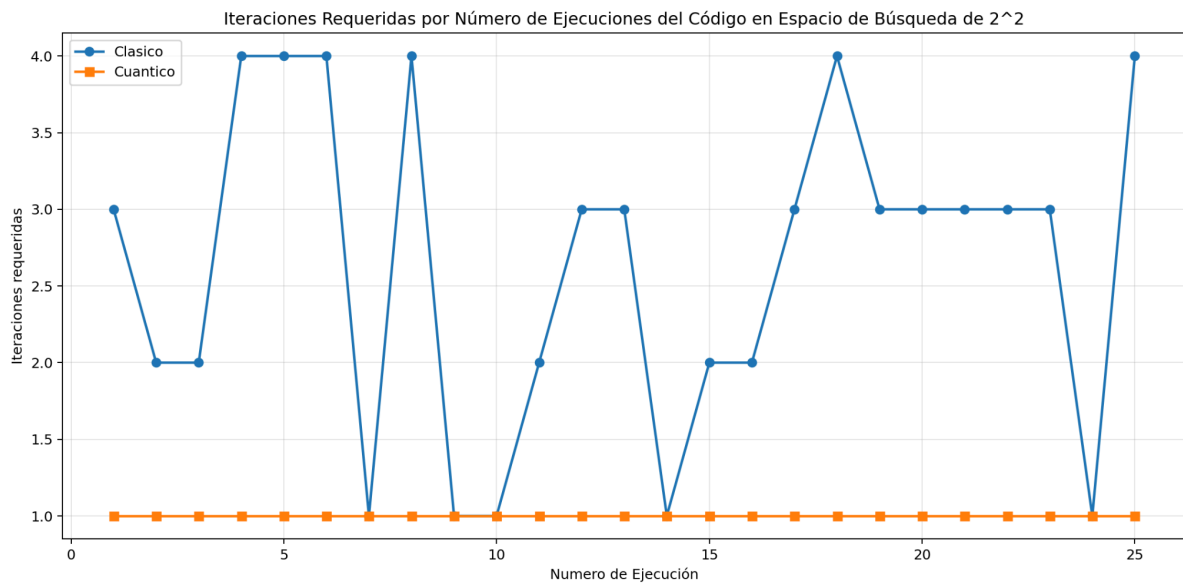


Figura C.1 - Iteraciones $n=2$

Modelo	Iteraciones promedio	Tiempo promedio
Clásico	2.64	0.000002
Cuántico	1.00	0.375563

Tabla C.1 - Resultados espacio 2^2

C.2. n=6:

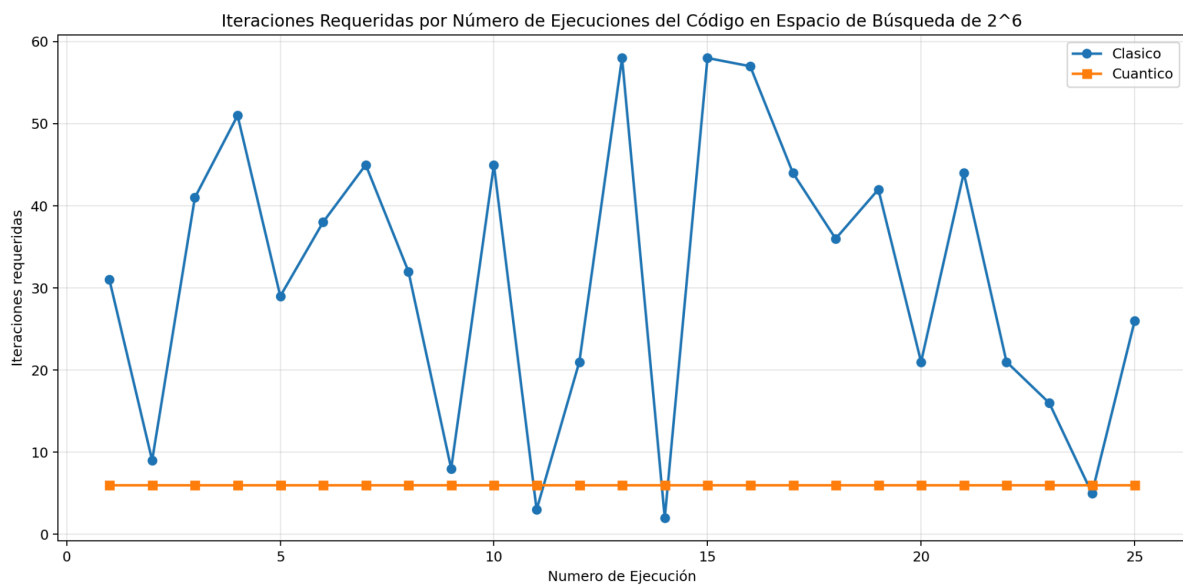


Figura C.2 - Iteraciones n=6

Modelo	Iteraciones promedio	Tiempo promedio
Clásico	31.32	0.000003
Cuántico	6.00	2.168292

Tabla C.2 - Resultados espacio 2^6

C.3. n=8:

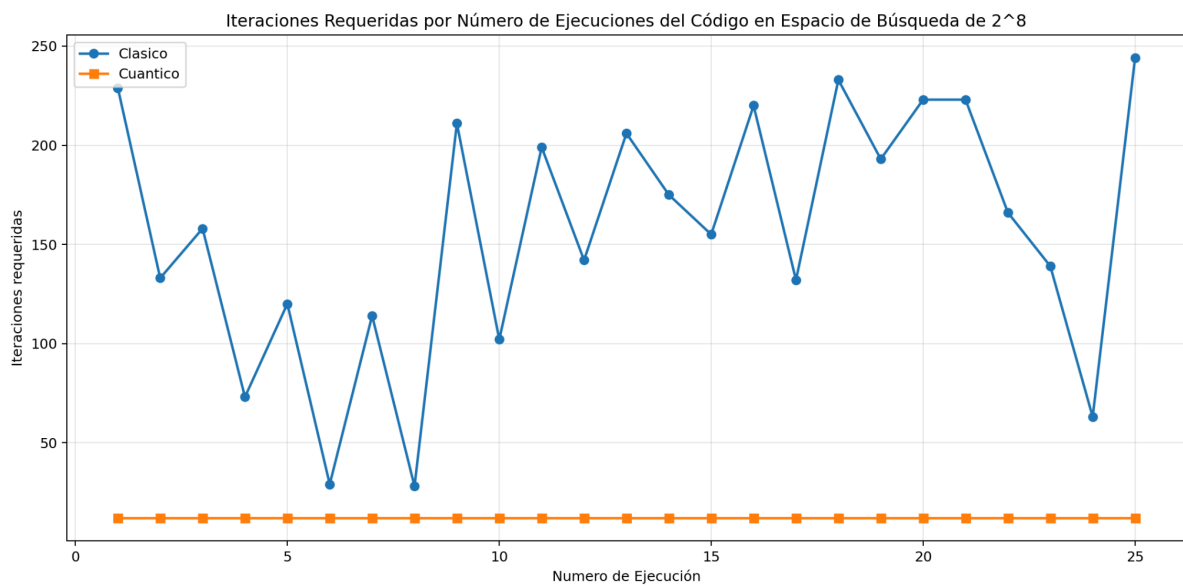


Figura C.3 - Iteraciones n=8

Modelo	Iteraciones promedio	Tiempo promedio
Clásico	156.40	0.000008
Cuántico	12.00	4.611284

Tabla C.3 - Resultados espacio 2^8

C.4. n=10:

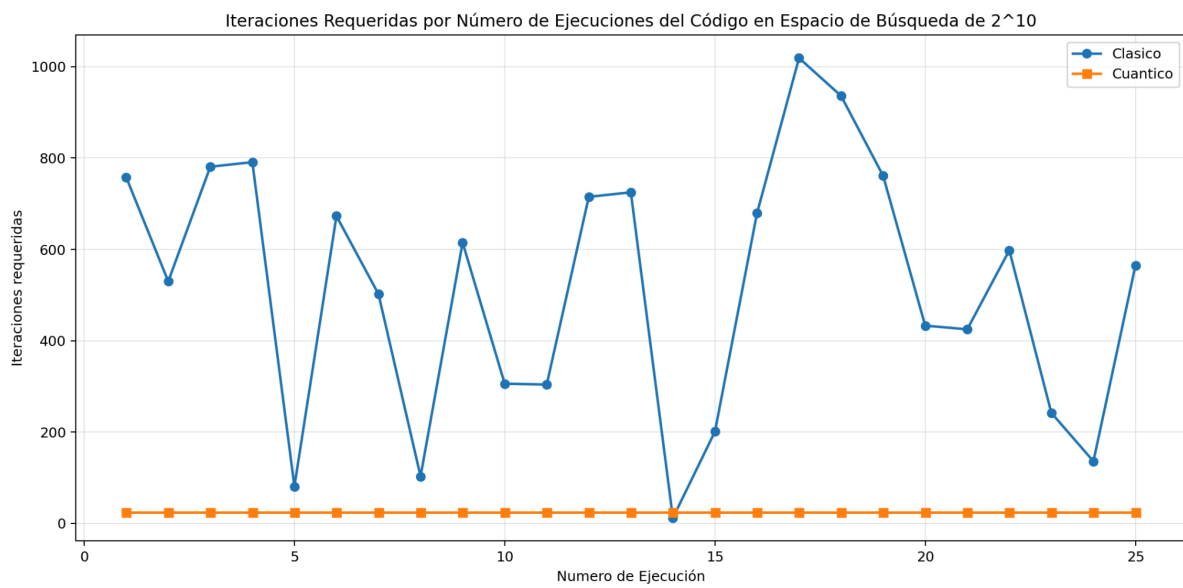


Figura C.4 - Iteraciones por ejecución

Modelo	Iteraciones promedio	Tiempo promedio
Clásico	515.64	0.000037 s
Cuántico	24.00	5.214571 s

Tabla C.4 - Resultados espacio 2^{10}

C.5. n=12:

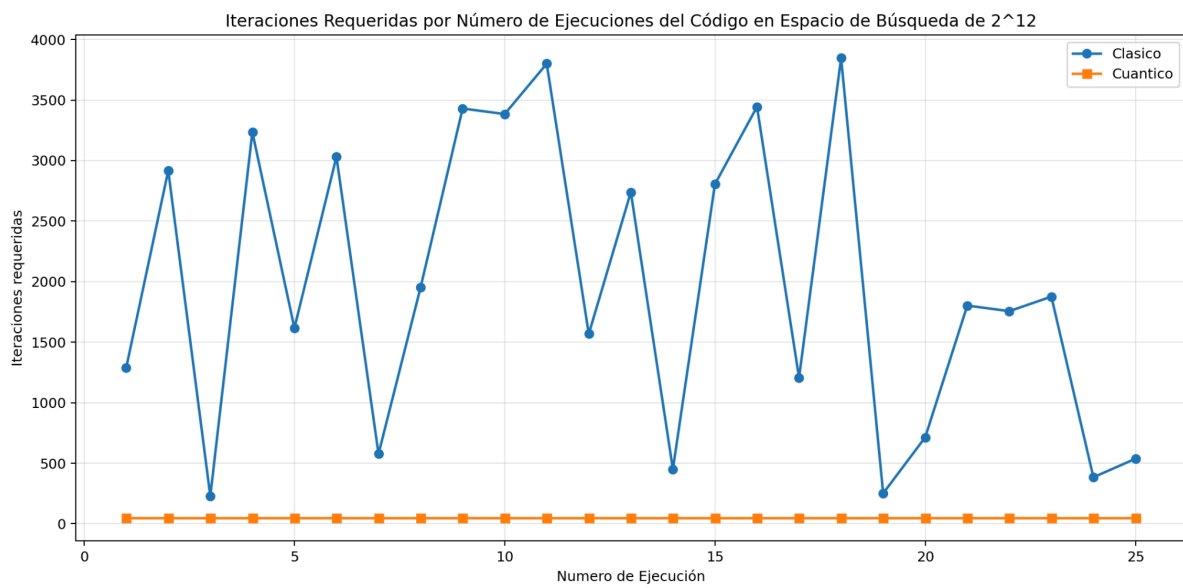


Figura C.5 - Iteraciones n=12

Modelo	Iteraciones promedio	Tiempo promedio
Clásico	1953.52	0.000138
Cuántico	48.00	7.112998

Tabla C.5 - Resultados espacio 2^{12}

[D] Resultados obtenidos durante el avance 2.

Link repositorio proyecto, creado durante este avance con archivos .csv, tablas e imágenes adicionales generadas: <https://github.com/Antix199/Q-Search>

D.1.

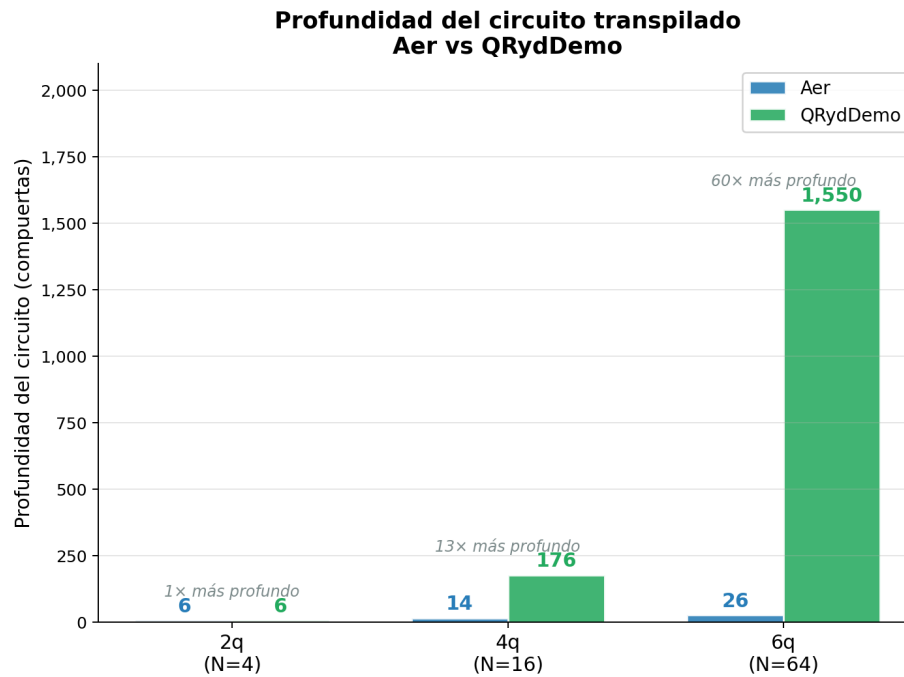


Figura D.1 - Profundidad del Circuito

D.2.

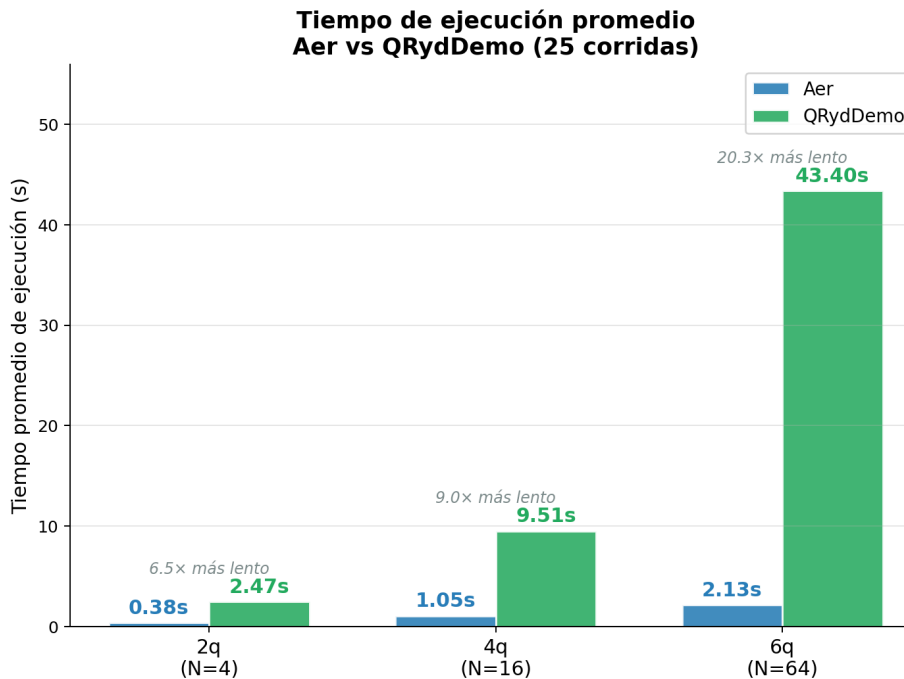


Figura D.2 - Tiempo de Ejecución Promedio

D.3.

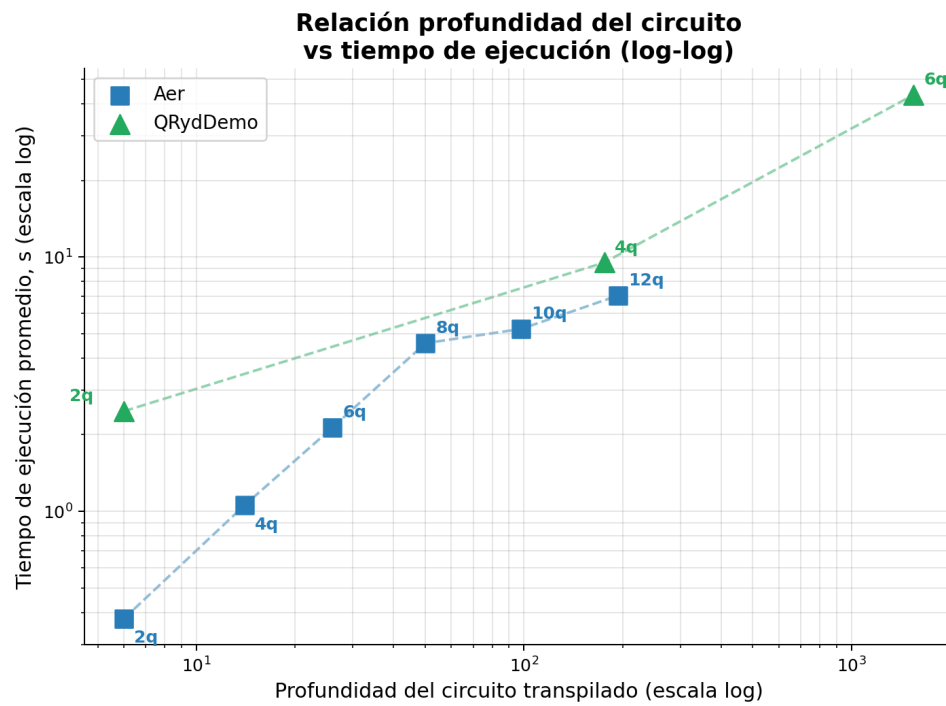


Figura D.3 - Profundidad del Circuito vs Tiempo de Ejecución

D.4.

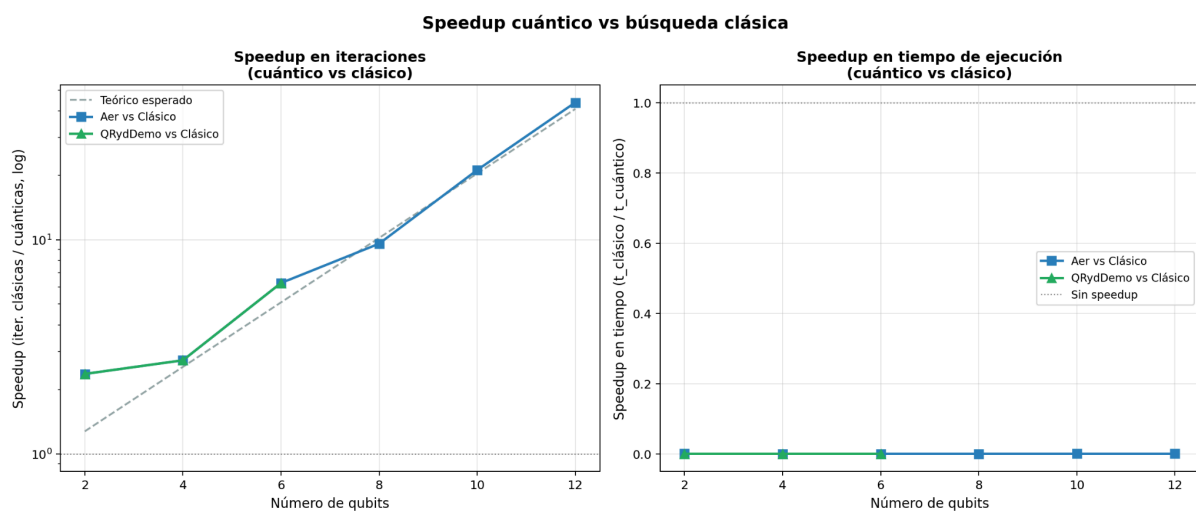


Figura D.4 - Speedup cuántico vs búsqueda clásica

D.5.

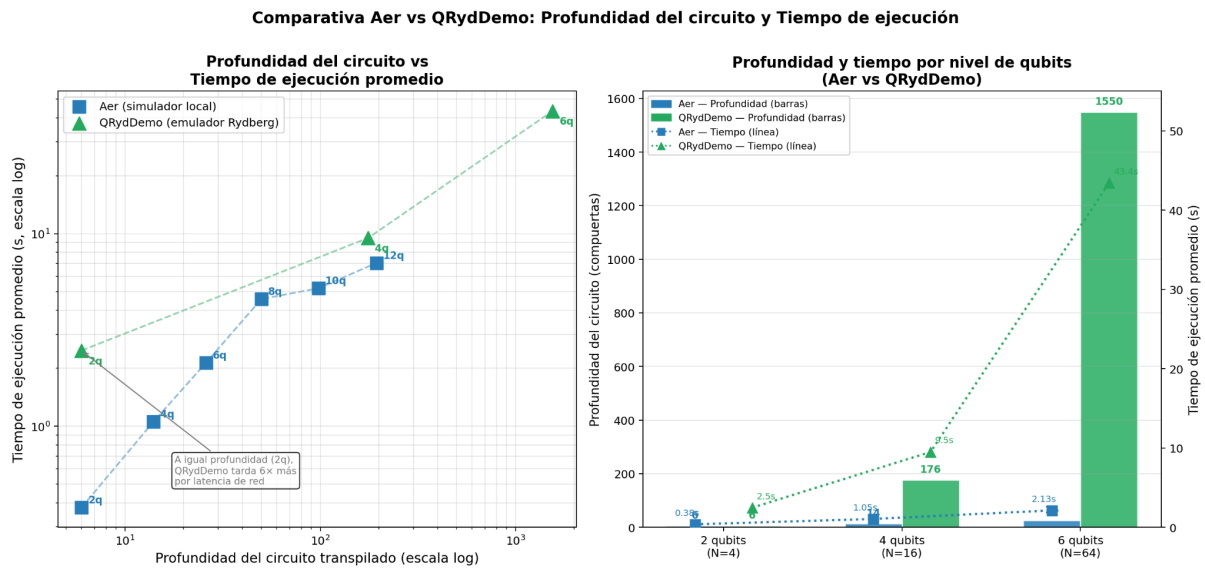


Figura D.5 - Profundidad del circuito y tiempo de ejecución

D.6.

Tiempo promedio de ejecución por nivel de qubits — Clásico vs Aer vs QRydDemo						
Qubits	N	Clásico (s)	Aer (s)	QRydDemo (s)	Aer / Clásico	QRyd / Clásico
2q	4	0.000001 s	0.3785 s	2.4743 s	378,476x	2,474,319x
4q	16	0.000001 s	1.0547 s	9.5056 s	1,054,667x	9,505,569x
6q	64	0.000002 s	2.1346 s	43.4034 s	1,067,280x	21,701,715x
8q	256	0.000005 s	4.5785 s	—	915,707x	—
10q	1,024	0.000031 s	5.2093 s	—	168,042x	—
12q	4,096	0.000148 s	7.0513 s	—	47,644x	—

Tabla D.1 - Tiempo de ejecución promedio por nivel de qubits

D.7. Tablas resultados por tipo de ejecución:

Bits	N	Corridas	Iter. Promedio	Mediana	Desv. Est.	Tiempo Prom. (s)	Iter. Esperadas (N/2)	Razón vs Esperado
2	4	25	2.36	2	0.95	1×10^{-6}	2	1.18
4	16	25	8.20	7	4.51	1×10^{-6}	8	1.03
6	64	25	37.64	43	19.38	2×10^{-6}	32	1.18
8	256	25	114.88	97	72.65	5×10^{-6}	128	0.90
10	1024	25	507.32	544	291.14	3.1×10^{-5}	512	0.99
12	4096	25	2098.04	2028	1185.11	1.48×10^{-4}	2048	1.02

Tabla D.2 - Resultados de búsqueda clásica

Bits	N	Iter. Prom.	Iter. Teórica	Razón	Éxito (%)	Profundidad	Evaluaciones	Speedup
2	4	1	1	1.00	100.00	6	2	2.36
4	16	3	3	1.00	97.34	14	6	2.73
6	64	6	6	1.00	99.78	26	12	6.27
8	256	12	12	1.00	100.00	50	24	9.57
10	1024	24	25	0.96	99.82	98	24	21.14
12	4096	48	50	0.96	99.76	194	24	43.71

Tabla D.3 - Resultados del algoritmo de Grover (Aer)

Bits	N	Iter. Prom.	Éxito (%)	Tiempo Prom. (s)	Profundidad	Compuertas	Entropía	Speedup
2	4	1	100.00	2.47	6	10	0.000	2.36
4	16	3	95.73	9.51	176	223.4	0.416	2.73
6	64	6	99.86	43.40	1550	2064	0.017	6.27

Tabla D.4 - Resultados en Qryd (hasta 6 qubits)